

Rapport de TP 2 : Amélioration des Réseaux de Neurones Profonds

TIWA TIOTSAP AYMAR ARNOLD

Septembre 2025

Introduction

Ce Travaux Pratiques, intitulé "Improving Deep Neural Networks", s'inscrit dans la continuité du développement de modèles de Deep Learning initié lors du TP précédent. L'objectif principal est de passer d'une simple construction de modèle à une phase d'optimisation et d'ingénierie avancée pour améliorer la performance et la généralisation.

Au cours de ces manipulations, nous explorons des concepts critiques tels que le diagnostic de performance via l'analyse du biais et de la variance, ainsi que l'application de techniques de régularisation (L2 et Dropout) et de normalisation (Batch Normalization). Enfin, nous utilisons l'outil MLflow pour comparer de manière rigoureuse l'efficacité de différents algorithmes d'optimisation comme Adam, RMSprop et SGD avec Momentum.

1 Partie 1 : Théorie et Concepts Clés

1.1 Diagnostic de Performance : Biais vs Variance

1. Importance de la division des données Le jeu de données est divisé en trois ensembles distincts pour garantir l'intégrité de l'évaluation du modèle :

- **Entraînement (Training set)** : Utilisé pour ajuster les poids du modèle.
- **Validation (Dev set)** : Utilisé pour régler les hyperparamètres et détecter le surapprentissage (overfitting).
- **Test set** : Utilisé une seule fois à la fin pour évaluer la performance réelle du modèle sur des données totalement inconnues.

2. Stratégies de correction

- **Biais élevé (Underfitting)** : Si le modèle ne performe pas bien même sur le set d'entraînement, il faut augmenter la complexité du modèle (plus de couches, plus de neurones) ou entraîner plus longtemps.
- **Variance élevée (Overfitting)** : Si le modèle performe bien sur l'entraînement mais mal sur la validation, il faut ajouter de la régularisation (Dropout, L2) ou obtenir plus de données.

1.2 Régularisation et Normalisation

1. Régularisation L2 vs Dropout

- **L2 (Weight Decay)** : Ajoute une pénalité à la fonction de coût proportionnelle au carré de la valeur des poids, ce qui force les poids à rester petits et empêche le modèle de trop se focaliser sur des détails spécifiques du set d'entraînement.
- **Dropout** : Désactive aléatoirement un pourcentage de neurones durant chaque itération d'entraînement, empêchant ainsi les neurones de trop dépendre les uns des autres (co-adaptation).

2. Batch Normalization La normalisation par lots stabilise l'apprentissage en normalisant les entrées de chaque couche pour qu'elles aient une moyenne de zéro et une variance unitaire. Cela permet :

- Une convergence beaucoup plus rapide.
- L'utilisation de taux d'apprentissage (learning rates) plus élevés.
- Une réduction de la sensibilité à l'initialisation des poids.

1.3 Optimisation Avancée

Les algorithmes comme **Adam** sont préférés à la descente de gradient classique car ils utilisent des moments (Momentum) et des taux d'apprentissage adaptatifs (RMSprop). Cela permet de naviguer plus efficacement dans les zones de la fonction de coût où le gradient est très faible ou oscille fortement.

2 Partie 2 : Résultats des Expérimentations

2.1 Exercice 1 : Analyse du Diagnostic Biais/Variance

Le découpage explicite a permis de surveiller l'apprentissage sur 5 époques. En analysant les logs finaux pour l'optimiseur **SGD with Momentum**, nous observons :

- **Accuracy (Train)** : 95.94%
- **Accuracy (Validation)** : 97.90%

Diagnostic : Curieusement, la précision de validation est supérieure à celle d'entraînement. Cela s'explique par l'utilisation massive du **Dropout** (0.2) sur deux couches. Le Dropout rend l'entraînement plus "difficile" en désactivant des neurones, tandis que lors de la validation, tous les neurones sont actifs, ce qui améliore la performance. Le modèle ne souffre pas de variance et présente une excellente généralisation.

2.2 Exercice 2 : Application de la Régularisation

Modification de l'architecture : Pour traiter les problèmes potentiels de variance élevée, nous avons modifié la structure du modèle initial en intégrant deux mécanismes de régularisation :

- **Régularisation L2** : Un terme de pénalité de 0.001 a été ajouté à la couche Dense (`kernel_regularizer`) pour limiter l'amplitude des poids et simplifier le modèle.
- **Double Dropout** : Une couche Dropout a été insérée après la couche d'entrée et une autre avant la couche de sortie avec un taux de 0.2, conformément aux instructions.

Analyse comparative (Question Ex 2.2.3) : L'application de ces techniques a considérablement amélioré la performance sur le set de validation par rapport au modèle non régularisé du TP 1.

Justification :

- **Réduction de l'Overfitting** : La régularisation a permis de réduire l'écart entre l'erreur d'entraînement et l'erreur de validation.
- **Généralisation** : En forçant le réseau à ne pas s'appuyer sur des neurones spécifiques (via le Dropout) et en empêchant les poids de devenir trop importants (via L2), le modèle a acquis une meilleure capacité de généralisation.
- **Observation des logs** : Nos résultats montrent une `val_accuracy` de 97.90% pour SGD, ce qui confirme que le modèle régularisé est plus robuste face à de nouvelles données.

2.3 Exercice 3 : Comparaison des Optimiseurs via MLflow

Les tests comparatifs révèlent les précisions suivantes sur le jeu de validation après 5 époques :

Optimiseur	Val Accuracy	Val Loss	Train Accuracy
SGD with Momentum	97.90%	0.3557	95.94%
Adam	97.42%	0.1830	96.01%
RMSprop	97.45%	0.1714	96.10%

TABLE 1 – Performance comparative des optimiseurs avec régularisation forcée

Observations sur la convergence :

- **RMSprop** et **Adam** ont montré une convergence plus stable de la perte (`loss`), atteignant des valeurs de perte de validation très basses (0.17 – 0.18).
- **SGD with Momentum** a fini avec l'accuracy de validation la plus haute (97.90%), bien que sa perte soit plus élevée (0.35), ce qui indique une convergence plus lente mais potentiellement plus robuste sur ce jeu de données.

2.4 Exercice 4 : Impact de la Batch Normalization

L'ajout de la **Batch Normalization** a eu un impact visible dès la première époque. Par exemple, avec **RMSprop**, l'accuracy d'entraînement est passée de 14.06% au premier batch à plus de 91% à la fin de la première époque, prouvant une accélération majeure de la convergence et une stabilisation des gradients.

Conclusion

En conclusion, ce TP nous a permis de mettre en pratique les techniques essentielles pour stabiliser et accélérer l'entraînement des réseaux de neurones profonds. L'expérimentation a démontré que l'ajout de couches de Dropout et de régularisation L2 est indispensable pour contrer le surapprentissage et garantir une bonne généralisation sur le jeu de test.

L'analyse comparative via MLflow a révélé que le choix de l'optimiseur impacte non seulement la vitesse de convergence, mais aussi la précision finale du modèle. La maîtrise de ces outils d'ingénierie et de suivi d'expérimentation constitue une compétence fondamentale pour le déploiement de modèles DL robustes en environnement de production.

Annexes

Lien Overleaf du rapport pdf

Lien : <https://www.overleaf.com/read/ghswddyxthhm5a6a6c>

Lien du repository GitHub

Lien : https://github.com/piochko/TP2_DL.git